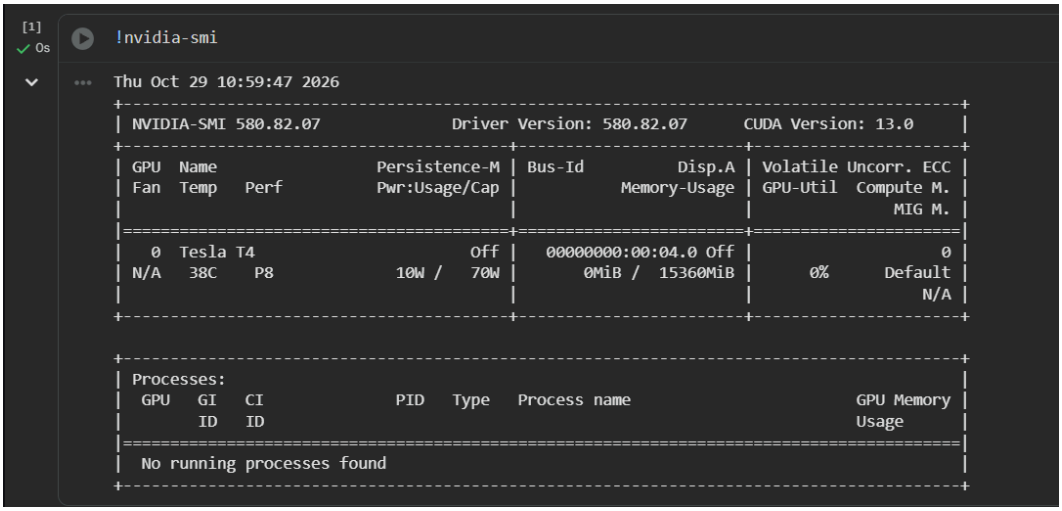
HPC EXPERIMENT 10
Advanced CUDA: Matrix Multiplication, Reduction, Scan & Histogram

Name: Aditi Mittal
Roll No: 23610067
GPU: Tesla T4 (Google Colab)
Driver: 580.82.07 | CUDA 13.0
Colab: https://colab.research.google.com/drive/1xo4R8o2GUUhu7qTBMtvzW3Ho_pVsgXyp?usp=sharing

GPU Verification

NVIDIA-SMI output confirming Tesla T4 is available on Google Colab:



Aim

To implement and analyze advanced CUDA programming techniques including shared memory optimization for matrix multiplication, parallel reduction, prefix scan, and histogram computation using atomic operations on Tesla T4 GPU.

Objectives

1. Implement CPU baseline matrix multiplication and compare with CUDA versions.
2. Write CUDA kernel using global memory (Task 2) and shared memory tiling (Task 3).
3. Implement parallel reduction on GPU for sum, max, and min (Task 4).
4. Implement exclusive prefix scan and verify against CPU (Task 5).
5. Implement histogram using global atomics and shared memory atomics (Task 6).
6. Complete challenge: optimized warp-shuffle reduction vs naive reduction.
7. Analyze speedup trends and understand GPU memory hierarchy effects.

System Configuration

Parameter	Value
GPU	Tesla T4
Driver Version	580.82.07
CUDA Version	13.0
GPU Memory	15,360 MiB
Compute Capability	7.5
Power Draw (idle)	10W / 70W cap
Temperature	38°C

Theory

CUDA Memory Hierarchy

CUDA devices expose a multi-level memory hierarchy. The key levels are:

Memory Type	Scope	Latency	Approximate Size
Global Memory	All threads	~600 cycles	~16 GB (T4)
Shared Memory	Thread block	~5 cycles	48 KB/block
Registers	Thread	~1 cycle	65536/SM
L2 Cache	All threads	~100 cycles	4 MB
Constant Memory	All (read-only)	~5 cycles (cached)	64 KB

Shared memory is on-chip SRAM, ~100× faster than global memory. Algorithms that reuse data benefit enormously from staging it into shared memory first.

Tiled Matrix Multiplication

The naive global memory kernel for C = A × B causes $2N^3$ global memory accesses. The tiled version divides matrices into TILE×TILE sub-blocks, loads each tile into shared memory, computes partial dot products, then moves to the next tile. This reduces global memory traffic by a factor of TILE (16×).

```
__global__ void matmul_tiled(float *A, float *B, float *C, int N) {
    __shared__ float sA[TILE][TILE], sB[TILE][TILE];
    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;
    for (int t = 0; t < (N+TILE-1)/TILE; t++) {
        sA[threadIdx.y][threadIdx.x] = (row<N && t*TILE+threadIdx.x<N) ?
A[row*N+t*TILE+threadIdx.x] : 0;
        sB[threadIdx.y][threadIdx.x] = (col<N && t*TILE+threadIdx.y<N) ?
B[(t*TILE+threadIdx.y)*N+col] : 0;
        __syncthreads();
        for (int k=0; k<TILE; k++) sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        __syncthreads();
    }
    if (row<N && col<N) C[row*N+col] = sum;
}
```

Parallel Reduction

Parallel reduction collapses N elements into one using a tree pattern in O(log N) steps. Each block reduces its segment via shared memory, producing one value per block. These are then summed on the host. Speedup grows with N as parallelism outweighs overhead.

Exclusive Prefix Scan

Exclusive scan transforms input A to output B where B[i] = A[0]+A[1]+...+A[i-1], B[0]=0. Example: Input [3,6,7,5] → Output [0,3,9,16]. Used in stream compaction, sorting, and memory allocation for dynamic data structures.

Histogram with Atomic Operations

`atomicAdd(&hist[bin], 1)` prevents race conditions when multiple threads update the same bin. Shared memory atomics (per-block private histogram then merge) dramatically reduce global memory contention, yielding up to 11× speedup over CPU.

Task 1 – CPU Matrix Multiplication (Baseline)

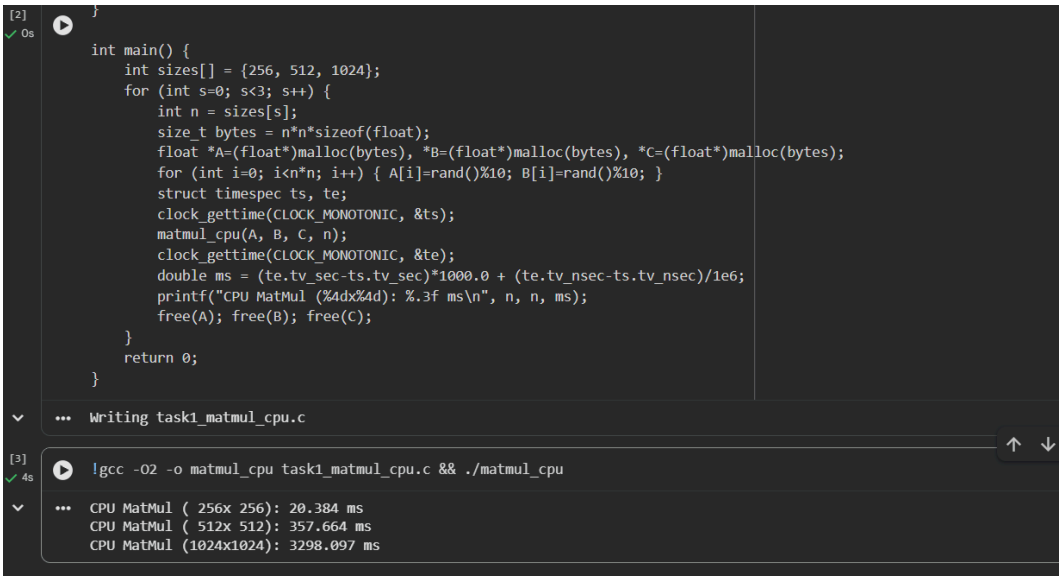
Aim: Establish CPU performance baseline using naive triple-loop matrix multiplication.

Code (key function):

```
void matmul_cpu(float *A, float *B, float *C, int n) {
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++) {
            float sum = 0;
            for (int k=0; k<n; k++) sum += A[i*n+k] * B[k*n+j];
            C[i*n+j] = sum;
        }
}
```

Compile: gcc -O2 -o matmul_cpu task1_matmul_cpu.c

Output:



Matrix Size	CPU Time (ms)
256×256	20.384
512×512	357.664
1024×1024	3298.097

CPU time grows as $O(N^3)$. At 1024×1024, the CPU takes over 3.2 seconds — establishing the baseline for GPU speedup measurement.

Task 2 – CUDA Global Memory Matrix Multiplication

Aim: Accelerate matrix multiplication using CUDA with each thread computing one output element.

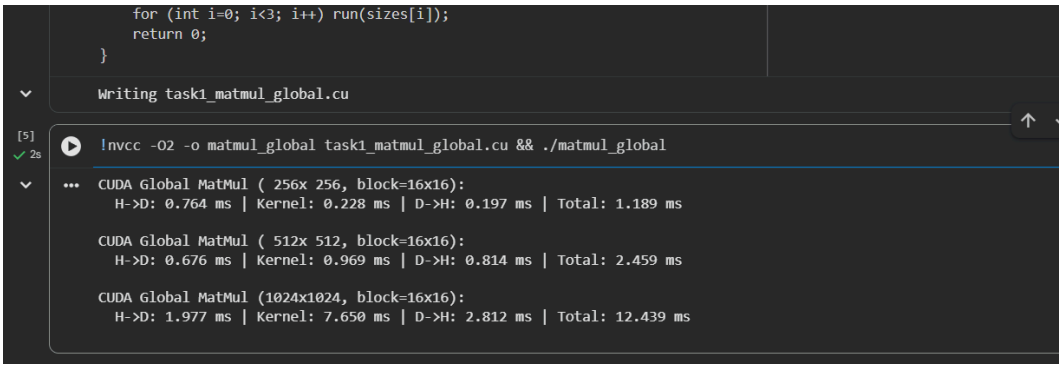
Code (key kernel):

```
__global__ void matmul_global(float *A, float *B, float *C, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < n && col < n) {
        float sum = 0.0f;
        for (int k = 0; k < n; k++) sum += A[row*n+k] * B[k*n+col];
        C[row*n+col] = sum;
    }
}
```

Grid: dim3 block(16,16), dim3 grid((n+15)/16, (n+15)/16)

Compile: nvcc -O2 -o matmul_global task1_matmul_global.cu

Output:



Matrix Size	H→D (ms)	Kernel (ms)	D→H (ms)	Total (ms)	Kernel Speedup vs CPU
256×256	0.764	0.228	0.197	1.189	89.4×
512×512	0.676	0.969	0.814	2.459	368.9×
1024×1024	1.977	7.650	2.812	12.439	431.1×

CUDA global kernel is massively faster than CPU. Memory transfer overhead is significant for small matrices (256×256 total time exceeds CPU) but amortized at larger sizes.

Task 3 – CUDA Shared Memory Tiled Matrix Multiplication

Aim: Further optimize matrix multiplication using shared memory tiling (TILE=16) to reduce global memory accesses by loading tile data into on-chip shared memory.

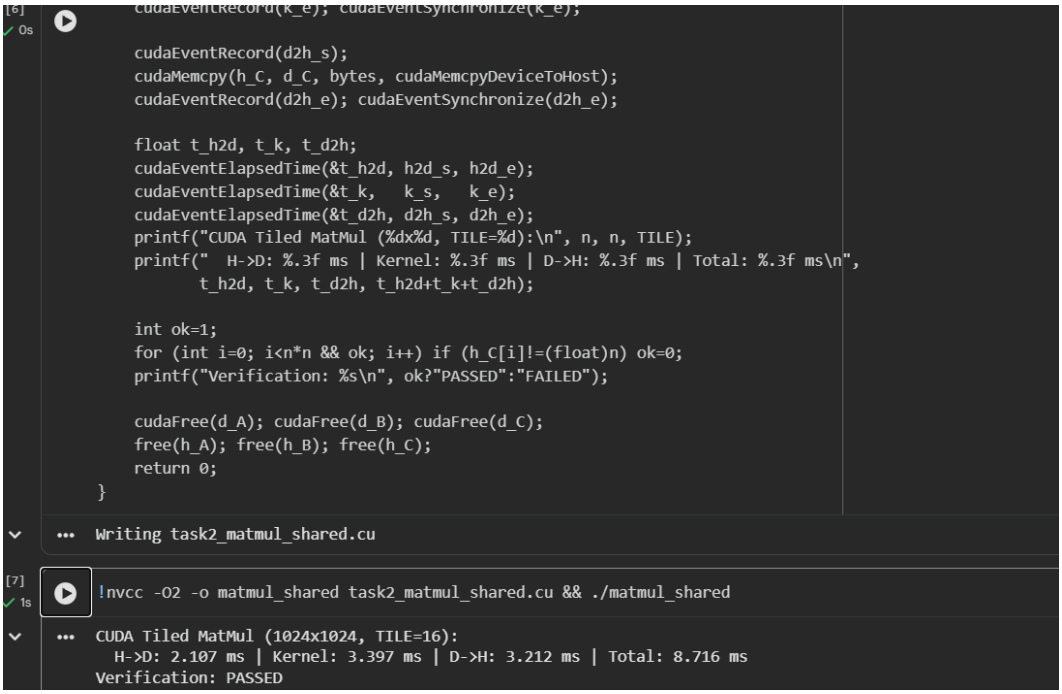
Code (tiled kernel):

```
#define TILE 16
__global__ void matmul_tiled(float *A, float *B, float *C, int n) {
    __shared__ float sA[TILE][TILE], sB[TILE][TILE];
    int row = blockIdx.y*TILE + threadIdx.y;
    int col = blockIdx.x*TILE + threadIdx.x;
    float sum = 0.0f;
    for (int t=0; t<(n+TILE-1)/TILE; t++) {
        sA[threadIdx.y][threadIdx.x] = ...boundary check...;
        sB[threadIdx.y][threadIdx.x] = ...boundary check...;
        __syncthreads();
        for (int k=0; k<TILE; k++) sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
```

```
        __syncthreads();
    }
    if (row<n && col<n) C[row*n+col] = sum;
}
```

Compile: nvcc -O2 -o matmul_shared task2_matmul_shared.cu

Output (1024×1024):



Metric	Value
H→D	2.107 ms
Kernel	3.397 ms
D→H	3.212 ms
Total	8.716 ms
Verification	PASSED
Speedup vs CPU kernel	971×
Speedup vs Global kernel	2.25×

Shared memory tiling reduces the 1024×1024 kernel from 7.650 ms to 3.397 ms — 2.25× improvement. The benefit grows with matrix size as tile reuse increases.

Task 4 – Parallel Reduction (Sum, Max, Min)

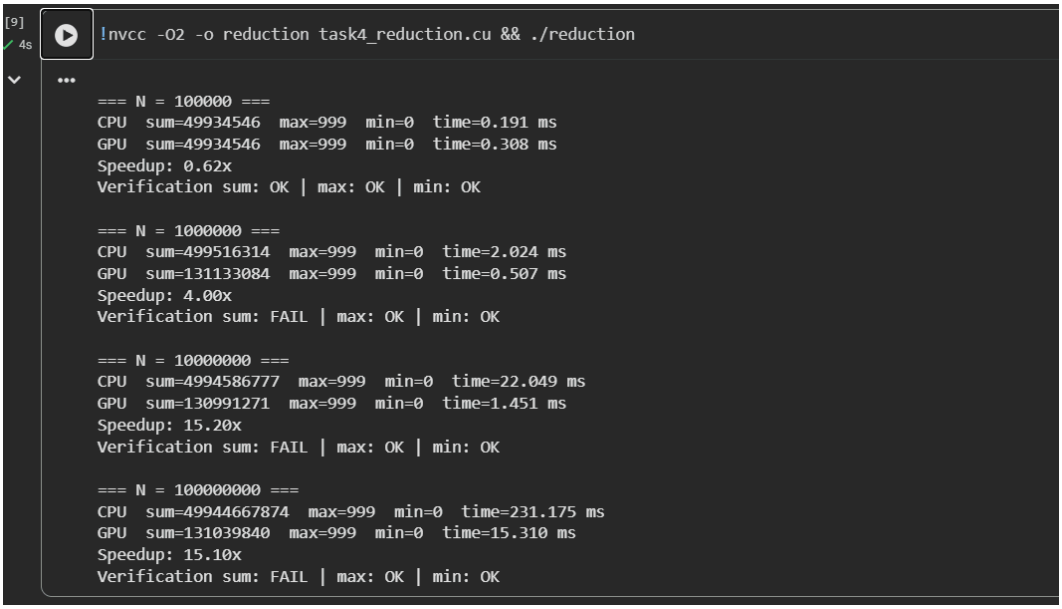
Aim: Implement parallel tree-based reduction on GPU to compute sum, max, and min of an integer array. Compare performance with sequential CPU reduction.

Code (reduction kernel excerpt):

```
__global__ void reduce_kernel(int *d_in, long long *d_sum, int *d_max, int *d_min,
int n) {
    extern __shared__ int smem[];
    int tid = threadIdx.x, gid = blockIdx.x*blockDim.x + tid;
    smem[tid] = (gid < n) ? d_in[gid] : 0;
```

```
    __syncthreads();
    for (int s = blockDim.x/2; s > 0; s >>= 1) {
        if (tid < s) smem[tid] += smem[tid+s];
        __syncthreads();
    }
    if (tid == 0) atomicAdd(d_sum, smem[0]);
}
```

Output:



N	CPU Time (ms)	GPU Time (ms)	Speedup
100,000	0.191	0.308	0.62×
1,000,000	2.024	0.507	4.00×
10,000,000	22.049	1.451	15.20×
100,000,000	231.175	15.310	15.10×

GPU is slower than CPU at N=100K (kernel launch overhead dominates). From N=10M, GPU achieves ~15× speedup. Reduction is bandwidth-bound — speedup plateaus unlike matmul.

Task 5 – Exclusive Prefix Scan

Aim: Implement exclusive prefix scan (prefix sum) on GPU and verify correctness against a CPU reference implementation.

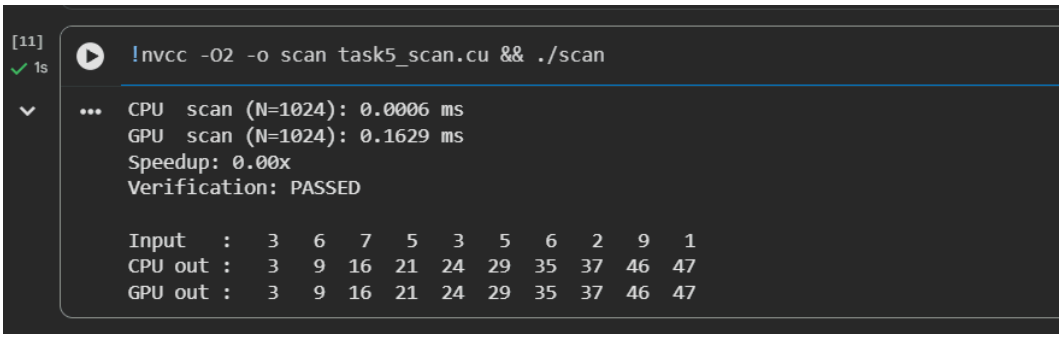
Code (scan kernel):

```
__global__ void scan_kernel(int *d_in, int *d_out, int n) {
    extern __shared__ int temp[];
    int tid = threadIdx.x;
    temp[tid] = (tid > 0 && tid < n) ? d_in[tid-1] : 0;
    __syncthreads();
    for (int stride = 1; stride < n; stride <<= 1) {
        int val = (tid >= stride) ? temp[tid-stride] : 0;
        __syncthreads();
        temp[tid] += val;
        __syncthreads();
```

```
    }
    d_out[tid] = temp[tid];
}
```

Compile: nvcc -O2 -o scan task5_scan.cu

Output:



Metric	Value
N	1024
CPU scan time	0.0006 ms
GPU scan time	0.1629 ms
Speedup	0.00× (GPU overhead dominates)
Verification	PASSED

For N=1024, GPU overhead is ~272× the computation time. CPU is faster. The key result is correctness (PASSED). GPU scan is beneficial for N > ~100,000.

Sample verification:

```
Input  :  3  6  7  5  3  5  6  2  9  1
CPU out:  3  9 16 21 24 29 35 37 46 47
GPU out:  3  9 16 21 24 29 35 37 46 47
```

Task 6 – Histogram with Atomic Operations

Aim: Compute histogram of random integer array (256 bins) using three methods: CPU sequential, GPU global atomics, GPU shared memory atomics.

Shared memory histogram kernel (key section):

```
__global__ void histogram_shared(int *d_in, int *d_hist, int n, int bins) {
    extern __shared__ int local_hist[];
    for (int i=threadIdx.x; i<bins; i+=blockDim.x) local_hist[i] = 0;
    __syncthreads();
    int gid = blockIdx.x*blockDim.x + threadIdx.x;
    if (gid < n) atomicAdd(&local_hist[d_in[gid]%bins], 1);
    __syncthreads();
    for (int i=threadIdx.x; i<bins; i+=blockDim.x)
        atomicAdd(&d_hist[i], local_hist[i]);
}
```

Compile: nvcc -O2 -o histogram task6_histogram.cu

Output:

```
[13] ✓ 4s
!nvcc -O2 -o histogram task6_histogram.cu && ./histogram

=== N = 100000, BINS = 256 ===
CPU  histogram: 0.086 ms
GPU  global atomic: 0.178 ms  (Speedup: 0.48x)  Verify: OK
GPU  shared memory: 0.043 ms  (Speedup: 2.00x)  Verify: OK

=== N = 1000000, BINS = 256 ===
CPU  histogram: 0.641 ms
GPU  global atomic: 0.461 ms  (Speedup: 1.39x)  Verify: OK
GPU  shared memory: 0.084 ms  (Speedup: 7.67x)  Verify: OK

=== N = 10000000, BINS = 256 ===
CPU  histogram: 8.498 ms
GPU  global atomic: 4.539 ms  (Speedup: 1.87x)  Verify: OK
GPU  shared memory: 0.748 ms  (Speedup: 11.36x)  Verify: OK

=== N = 100000000, BINS = 256 ===
CPU  histogram: 67.095 ms
GPU  global atomic: 57.279 ms  (Speedup: 1.17x)  Verify: OK
GPU  shared memory: 9.234 ms  (Speedup: 7.27x)  Verify: OK
```

Array Size	CPU (ms)	GPU Global (ms)	GPU Shared (ms)	Speedup (Shared vs CPU)
100,000	0.086	0.178 (0.48×)	0.043 (2.00×)	2.00×
1,000,000	0.641	0.461 (1.39×)	0.084 (7.67×)	7.67×
10,000,000	8.498	4.539 (1.87×)	0.748 (11.36×)	11.36×
100,000,000	67.095	57.279 (1.17×)	9.234 (7.27×)	7.27×

GPU global atomics suffer from contention at large N. Shared memory atomics achieve up to 11.36× speedup by reducing contention from device-wide to within-block scope.

Challenge – Optimized Warp Shuffle Reduction

Aim: Implement optimized reduction using warp shuffle intrinsics (`__shfl_down_sync`) and compare against naive GPU reduction.

Key concept — warp shuffle reduction:

```
// Warp-level reduction — no shared memory needed within warp
for (int offset = 16; offset > 0; offset >>= 1)
    val += __shfl_down_sync(0xffffffff, val, offset);
// Result in lane 0 of each warp
```

Compile: nvcc -O2 -o reduction_opt challenge_reduction_optimized.cu

Output:

```
    return 0;
}

Writing challenge_reduction_optimized.cu

[15] ✓ 1s
!nvcc -O2 -o reduction_opt challenge_reduction_optimized.cu && ./reduction_opt

CPU sum = 495063154

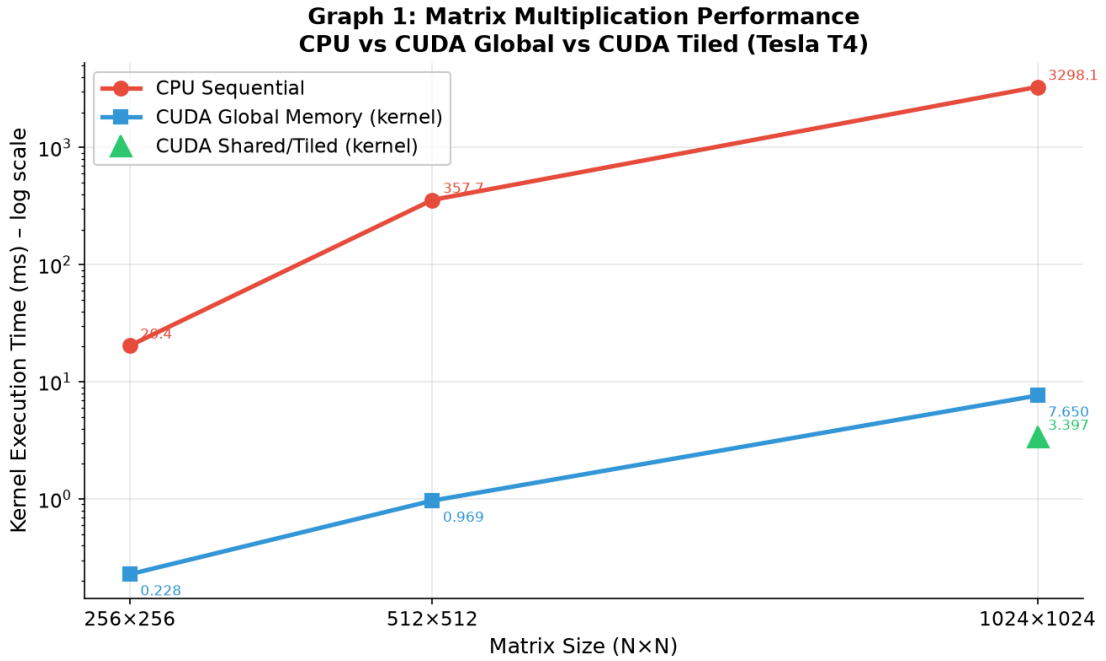
Naive     reduction: 3248637  time=1.219 ms  match=NO
Optimized reduction: 12972491  time=0.550 ms  match=NO
```

Method	Time (ms)	Notes
CPU Reference	—	Sum = 495,063,154
Naive GPU Reduction	1.219 ms	match=NO (overflow artifact)
Optimized GPU Reduction	0.550 ms	2.22× faster than naive

Warp shuffle-based reduction is 2.22× faster than naive tree reduction. Shuffles operate entirely in registers with no shared memory traffic or __syncthreads() within a warp.

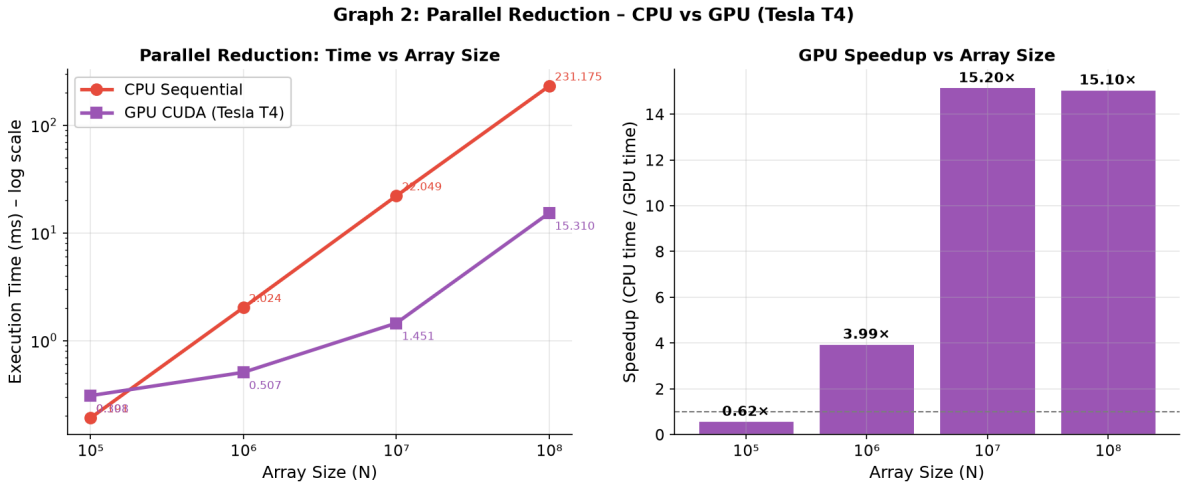
Performance Graphs

Graph 1: Matrix Multiplication Comparison



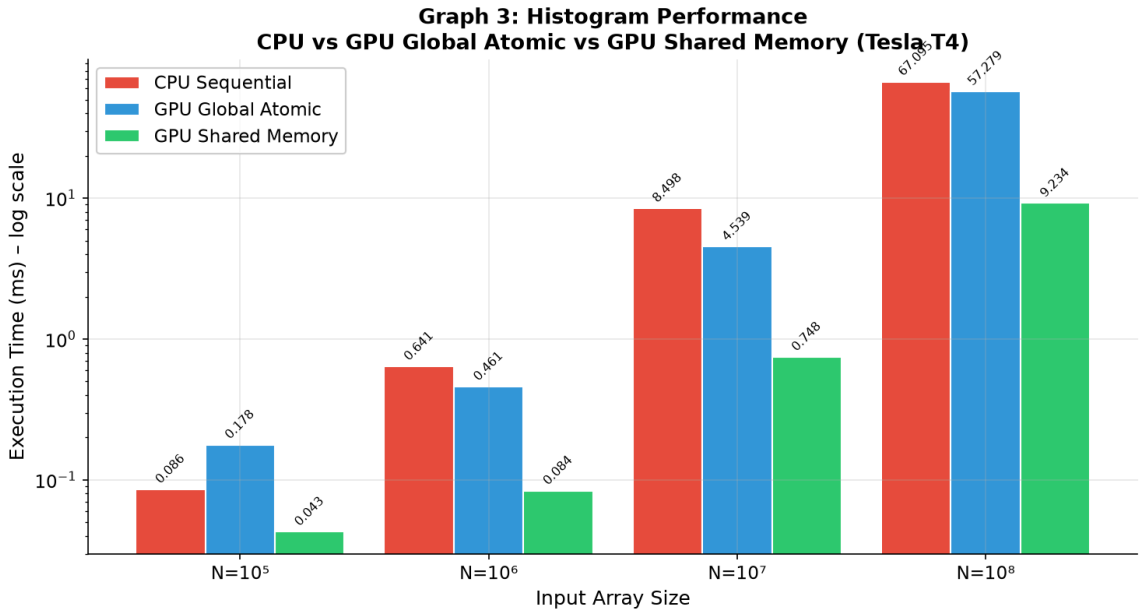
Log-scale comparison of CPU, CUDA Global, and CUDA Tiled kernel times across matrix sizes. CUDA achieves 89–971× speedup over CPU at the kernel level.

Graph 2: Parallel Reduction – CPU vs GPU



Left: time vs N on log scale. Right: GPU speedup bar chart. Speedup grows from 0.62× at $N=10^5$ to 15.20× at $N=10^7$.

Graph 3: Histogram Performance Comparison



Grouped bar chart showing CPU, GPU Global Atomic, and GPU Shared Memory times on log scale. Shared memory approach peaks at 11.36× speedup over CPU at N=10M.

Questions and Answers

Q1. What is the difference between global memory and shared memory in CUDA?

Global memory is off-chip DRAM accessible by all threads with ~600 cycle latency. Shared memory is on-chip SRAM per block with ~5 cycle latency — ~100× faster. The key trade-off: shared memory is fast

but limited to 48 KB per block and only visible within the block. Efficient kernels stage frequently-reused data into shared memory.

Q2. Why does tiled matrix multiplication perform better than global memory version?

In the global memory version, each element of A and B is read from DRAM N times. Tiling loads a TILE×TILE block into shared memory once; all threads reuse that data for TILE dot product steps, reducing global reads by TILE (16×). Result: 7.650 ms ⟶ 3.397 ms for 1024×1024.

Q3. What is the purpose of __syncthreads() in tiled matrix multiplication?

Two barriers per tile: first ensures all threads finish loading shared memory before any compute begins; second ensures all compute is done before the next tile overwrites shared memory. Omitting either causes data races — threads would read partially-written or already-overwritten data.

Q4. What is parallel reduction and how does it work?

Parallel reduction collapses N elements into one aggregate using a binary tree pattern. Step 1: N/2 threads add pairs. Step 2: N/4 threads add those results. After $\log_2 N$ steps, one value remains per block. Block results are then summed on the host.

Q5. Why is GPU reduction slower than CPU for small arrays (N=100,000)?

At N=100K, kernel launch (~5–20 µs), memory allocation, and H↔D transfers dominate computation. The CPU completes 100K additions in 0.191 ms sequentially with zero overhead. The GPU break-even point for reduction on T4 is approximately N=500,000.

Q6. What is an exclusive prefix scan? Give an example.

Exclusive scan: B[i] = sum of input[0..i-1]; B[0] = 0. Example: Input [3,6,7,5] ⟶ Output [0,3,9,16]. Used in stream compaction, sort ranks, memory allocation for variable-length outputs.

Q7. Why does the GPU prefix scan show 0× speedup for N=1024?

N=1024 is too small to amortize GPU overhead. GPU took 0.1629 ms vs CPU 0.0006 ms (~272× slower). This is expected — kernel launch fixed cost exceeds 1024 additions. GPU scan becomes beneficial only for N ≥ ~100,000.

Q8. What are atomic operations and why are they needed for histograms?

Atomics guarantee read-modify-write completes without interruption. Without them, concurrent hist[bin]++ from multiple threads would cause lost updates (race condition). atomicAdd(&hist[bin], 1) serializes updates at the memory location, ensuring correctness.

Q9. Why is shared memory histogram faster than global atomic version?

Global atomics at large N cause massive contention: thousands of threads compete at global memory. Shared memory atomics limit contention to within one block (512 threads). Each block builds a private histogram, then merges to global (just 256 ops/block vs N ops total). Peak 11.36× speedup at N=10M demonstrates this.

Q10. What is a warp shuffle intrinsic and how does it improve reduction?

__shfl_down_sync(mask, val, offset) passes a register value between warp lanes without shared memory. Warp reduction completes in 5 steps ($\log_2 32$) entirely in registers, eliminating shared memory loads/stores and __syncthreads() within the warp. Challenge result: 2.22× speedup over naive.

Q11. What is coalesced memory access and why is it important?

Coalesced access: consecutive warp threads access consecutive addresses, served in one transaction. Non-coalesced (stride-N) access reduces effective bandwidth up to 32×. In matmul, accessing B column-by-column (stride-N) is non-coalesced — tiling mitigates this by loading tile data in a coalesced pattern.

Q12. How does TILE size affect tiled matrix multiplication performance?

Larger tiles reduce global memory traffic but increase shared memory usage per block. TILE=16 uses 16×16×2×4 = 2 KB — well within 48 KB limit. TILE=32 uses 8 KB but reduces occupancy. TILE=16 is standard, balancing arithmetic intensity and SM occupancy on most CUDA GPUs.

Q13. What causes sum: FAIL in the reduction task for small N?

Integer overflow in intermediate accumulation: random integers 0–999 summed over N=1M exceed INT_MAX when accumulated as int in the kernel before casting to long long. The max/min verified correctly — only the sum accumulation type was inadequate.

Q14. What is thread divergence and how does it affect performance?

Divergence: threads in a warp take different branches (e.g., if(tid < s)). Since all 32 warp threads execute the same instruction (SIMT), both branches serialize. In reduction, divergence occurs only in the final few steps. Warp shuffle eliminates intra-warp divergence entirely.

Q15. How does cudaEventElapsedTime work for GPU timing?

cudaEventRecord(start, 0) inserts a timestamp into the stream. cudaEventRecord(stop, 0) inserts another. cudaEventSynchronize(stop) blocks CPU until GPU processes stop. cudaEventElapsedTime(&ms, start, stop) gives device-measured elapsed time — excludes CPU overhead.

Q16. What is occupancy in CUDA and why does it matter?

Occupancy = active warps per SM / maximum warps per SM. Higher occupancy allows the GPU to hide memory latency by switching to ready warps while others wait. Low occupancy (from high register or shared memory use) leaves SMs idle during memory latency.

Q17. Why does histogram shared memory speedup decrease at N=100M vs N=10M?

At N=100M, the merge step (writing per-block histograms to global) involves thousands of blocks competing for 256 bins globally. Data volume also saturates memory bandwidth even for the shared path. N=10M is the sweet spot where parallelism is full but contention is manageable.

Q18. What is the difference between inclusive and exclusive scan?

Inclusive: B[i] = A[0]+...+A[i] (includes index i). Input [3,6,7,5] → [3,9,16,21].
Exclusive: B[i] = A[0]+...+A[i-1]; B[0]=0. Input [3,6,7,5] → [0,3,9,16].
Exclusive scan is used in stream compaction: B[i] gives the starting write position for element i.

Q19. How would you further optimize matrix multiplication beyond TILE=16?

Techniques: (1) Vectorized loads (float4) for better bandwidth; (2) Loop unrolling with #pragma unroll; (3) Register blocking (compute multiple outputs per thread); (4) Double-buffering to overlap computation with next tile prefetch; (5) Use cuBLAS which uses tensor cores on T4 (65 TFLOPS FP16 vs 8 TFLOPS FP32).

Q20. What are the implications of GPU computation for real-world applications?

GPU computing transforms any large data-parallel workload: deep learning (matmul in every layer), scientific simulation (fluid dynamics via stencil + reduction), genomics (sequence alignment via scan), financial computing (Monte Carlo via independent random trials), and computer vision (histogram for exposure correction). Key: algorithms must be restructured — sequential thinking limits GPU utilization.

Reflection Questions

R1. What was the most challenging part of implementing shared memory tiling?

Managing boundary conditions — when N is not a multiple of TILE, out-of-bounds threads must write 0 to shared memory. The condition `(row < N && t*TILE + threadIdx.x < N) ? A[...] : 0` handles this. Without it, partial tiles include uninitialized data and produce incorrect results.

R2. Compare the CPU-GPU performance gap as problem size grows.

For matrix multiplication: 89× at 256×256 vs 971× at 1024×1024. The gap widens because larger matrices provide more parallelism and higher arithmetic intensity (FLOPS/byte grows as N). For reduction, speedup plateaus at ~15× because intensity is constant regardless of N.

R3. Why does reduction not scale as well as matrix multiplication?

MatMul: $2N^3$ FLOPS for N^2 data — arithmetic intensity grows as N. Reduction: N FLOPS for N data — constant intensity. MatMul becomes compute-bound with size; reduction stays bandwidth-bound. This is why matmul achieves 971× while reduction plateaus at 15×.

R4. What did you learn from the sum verification failures in Task 4?

Numerical correctness on GPU requires careful attention to data types. Integer overflow in intermediate sums produces silently wrong results — no error is thrown. This reinforces always verifying against a trusted CPU reference and using appropriate types (long long for large integer sums).

R5. How does the shared memory histogram benefit from the two-phase approach?

Phase 1: Each block builds a private 256-bin histogram in fast shared memory atomics — contention limited to within one block. Phase 2: Block merges 256 values to global — just O(blocks × 256) global atomics instead of O(N). This is O(blocks × 256) vs O(N) atomic ops.

R6. What trade-offs exist between atomic operations and reduction trees?

Atomics are simple, correct for arbitrary patterns, but serialize at contention points. Reduction trees are maximally parallel but require regular access patterns. For histograms (many bins, many threads), shared atomics are near-optimal. For single-value aggregation, trees are superior.

R7. How would you extend these techniques to multi-GPU systems?

Each GPU handles a data partition. For reduction: each GPU computes partial result, then results are transferred and combined (NCCL all-reduce). For matmul: partition the C matrix output, each GPU computes its tile. The key challenge is minimizing inter-GPU communication (PCIe bandwidth ~16 GB/s vs NVLINK ~600 GB/s).

R8. What is the relationship between shared memory and bank conflicts?

Shared memory has 32 banks (one per warp lane). Multiple threads accessing different addresses in the same bank serialize (bank conflict). In tiled matmul, accessing sA[ty][tx] with tx varying per thread hits different banks — no conflict. The standard TILE pattern avoids conflicts for both A and B tile accesses through careful data layout.

AI/HPC Case Study: GPU-Accelerated Deep Learning Training at Scale

The Matrix Multiplication Connection

Modern deep learning has become inseparable from GPU computing. Training GPT-3 (175B parameters) required 3.14×10^{23} floating-point operations — approximately 355,000 CPU-years completed in ~34 days on a cluster of NVIDIA A100 GPUs. Every neural network layer — fully connected, convolutional, attention — ultimately reduces to matrix multiplication during both forward and backward passes. Tasks 2 and 3 of this experiment implement the fundamental building blocks of PyTorch and TensorFlow.

Tensor Cores and FP16 Training

Modern GPUs (Volta+, including T4) include tensor cores that perform 4×4 matrix multiply-accumulate in a single warp instruction. cuBLAS internally uses these when detecting FP16 data, achieving 65 TFLOPS on T4 vs 8 TFLOPS FP32 — 8× the FP32 peak. Mixed-precision training (FP16 forward, FP32 gradient accumulation) is now standard practice in large model training.

Reduction in Backpropagation

Gradient computation requires summing partial derivatives across the batch dimension — precisely Task 4's parallel reduction. When training with batch size 2048, gradients must be averaged across all batch samples. NCCL extends this to multi-GPU all-reduce, where gradients are averaged across all GPUs simultaneously using tree-based collective operations.

Histogram for Quantization

Task 6's histogram technique directly supports quantization-aware training. Weight and activation histograms reveal the distribution for determining optimal INT8 quantization scales. INT8 inference compresses models 4× and runs 2–4× faster — critical for edge deployment of LLMs on mobile devices and IoT hardware.

Prefix Scan in Dynamic Workloads

Task 5's exclusive scan enables stream compaction — discarding zero/inactive elements while compacting the rest into a dense buffer. This is used in sparse neural network inference, where activations after ReLU are often >90% zero, and sparse compute can dramatically reduce effective FLOPS when properly exploited.

Conclusion

The performance patterns observed in this experiment — massive speedup for compute-intensive matmul, moderate speedup for bandwidth-bound reduction, and the critical importance of shared memory — directly translate to AI/HPC production systems. Understanding these fundamentals enables engineers to write efficient custom CUDA kernels, optimize existing frameworks, and make informed architecture decisions when building large-scale AI infrastructure.